

```

import java.util.*;

public class DataStructuresDemo {
    public static void main(String[] args) {

        // =====
        // 1. ArrayList
        // =====
        List<String> arrayList = new ArrayList<>();
        arrayList.add("Element");    // add element to end of list
        arrayList.get(0);            // return element at index 0
        arrayList.set(0, "New Element"); // replace element at index 0
        arrayList.contains("Element"); // check if list contains element
        arrayList.size();            // return list size
        arrayList.remove(0);         // remove element at index 0
        arrayList.clear();           // remove all elements

        // =====
        // 2. LinkedList
        // =====
        LinkedList<String> linkedList = new LinkedList<>();
        linkedList.add("Element A"); // add element at first
        linkedList.addFirst("First"); // add element at first
        linkedList.addLast("Last");  // add element at last
        linkedList.getFirst();        // return first element without remove it
        linkedList.getLast();         // return last element without remove it
        linkedList.pop();             // remove the first element added to the list
        linkedList.removeFirst();     // remove the first element and return it
        linkedList.removeLast();     // remove the last element and return it

        // =====
        // 3. Vector
        // =====
        Vector<Integer> vector = new Vector<>();
        vector.addElement(10);        // add element
        vector.elementAt(0);          // return element at index
        vector.firstElement();        // return first element
        vector.capacity();            // return current capacity
        vector.size();               // return number of elements
        vector.removeElement(10);     // remove first occurrence of element

        // =====
    }
}

```

```
// 4. Stack
```

```
// =====
```

```
Stack<Integer> stack = new Stack<>();
```

```
stack.push(100);          // push element to top
```

```
stack.peek();             // return top element without removing
```

```
stack.search(100);        // return position from top
```

```
stack.pop();              // remove and return top element
```

```
stack.empty();            // check if stack is empty
```

```
// =====
```

```
// 5. HashMap
```

```
// =====
```

```
Map<Integer, String> hashMap = new HashMap<>();
```

```
hashMap.put(1, "One");    // add or update key-value pair
```

```
hashMap.get(1);           // return value for key
```

```
hashMap.containsKey(1);   // check if key exists
```

```
hashMap.containsValue("One"); // check if value exists
```

```
hashMap.keySet();         // return set of keys
```

```
hashMap.values();         // return collection of values
```

```
hashMap.remove(1);        // remove pair by key
```

```
// =====
```

```
// 6. LinkedHashMap
```

```
// =====
```

```
LinkedHashMap<Integer, String> linkedHashMap = new LinkedHashMap<>();
```

```
linkedHashMap.put(2, "Two"); // add key-value pair with insertion order
```

```
linkedHashMap.get(2);        // return value for key
```

```
linkedHashMap.entrySet();    // return set of all pairs
```

```
linkedHashMap.keySet();      // return set of keys in order
```

```
linkedHashMap.remove(2);     // remove key-value pair
```

```
linkedHashMap.clear();       // remove all elements
```

```
// =====
```

```
// 7. HashSet
```

```
// =====
```

```
Set<String> hashSet = new HashSet<>();
```

```
hashSet.add("Unique");       // add unique element
```

```
hashSet.contains("Unique");  // check if element exists
```

```
hashSet.size();              // return number of elements
```

```
hashSet.isEmpty();           // check if set is empty
```

```
hashSet.remove("Unique");    // remove specified element
```

```
hashSet.clear();             // remove all elements
```

```

// =====
// 8. ArrayDeque
// =====
ArrayDeque<Double> arrayDeque = new ArrayDeque<>();
arrayDeque.addFirst(1.1);    // add element to front
arrayDeque.addLast(2.2);     // add element to end
arrayDeque.peekFirst();      // return first element without removing
arrayDeque.peekLast();       // return last element without removing
arrayDeque.pollFirst();       // remove and return first element
arrayDeque.pollLast();        // remove and return last element
arrayDeque.push(3.3);         // push element to front
arrayDeque.pop();             // remove and return front element

// =====
// 9. LinkedHashSet
// =====
LinkedHashSet<String> linkedHashSet = new LinkedHashSet<>();
linkedHashSet.add("Ordered"); // add element with insertion order
linkedHashSet.contains("Ordered");// check if element exists
linkedHashSet.size();          // return number of elements
linkedHashSet.isEmpty();       // check if set is empty
linkedHashSet.remove("Ordered"); // remove specified element
linkedHashSet.clear();         // remove all elements
}
}

```